TREINAMENTO CYPRESS

APOSTILA DO ALUNO · DIA 1

Treinamento Cypress

DIA 01
Fundamentos

DIA 02
Construindo uma suíte

DIA 03
Profissionalizando

```
$ npx cypress run

  Página de login
    ✓ deve carregar a tela de login
    ✓ deve permitir login com credenciais válidas de administrador
    ✓ deve exibir mensagem de erro ao informar senha inválida

  Cadastro de usuário
    ✓ deve cadastrar um usuário comum

  76 passing
```

CONTEÚDO

Sumário

APOSTILA

Treinamento Cypress 3

Para quem é esta apostila 3

Como usar 3

Antes do primeiro dia 3

A aplicação que vamos testar 4

Convenções desta apostila 4

Uma promessa sobre o terceiro dia 5

01 DIA 1

Fundamentos 6

1.1 Estratégia antes da ferramenta 6

1.2 JavaScript essencial 8

1.3 Instalação e configuração 11

1.4 Seu primeiro teste 20

1.5 Comandos e asserções 24

1.6 Hooks e independência de testes 27

1.7 Seletores e como se desbloquear sozinho 31

1.8 Lab do Dia 1 33

Resumo do Dia 1 34

APÊNDICES

Consulta rápida 35

A Comandos do Cypress por finalidade 35

B Tabela de asserções 37

C Como ler uma falha: o método de 4 passos 38

D Mensagens de erro e o que significam 38

E Seletores do ServeRest 39

APOSTILA

Treinamento Cypress

3 dias · Automação de testes com Cypress · ServeRest

Para quem é esta apostila

Para você, que está começando em automação de testes. Não presumimos que você saiba programar — presumimos que você saiba testar, e vamos somar a isso a capacidade de escrever o teste em código.

Se você vem de teste manual: **você já tem a parte difícil.** Saber o que validar, imaginar o cenário que ninguém pensou, desconfiar do caminho feliz — isso não se aprende em três dias. Sintaxe, sim.

Como usar

Esta apostila acompanha o treinamento, mas foi escrita para funcionar **depois dele**, quando você estiver sozinho na frente de um teste que não passa.

- **Em três partes, uma por dia.** Você recebe a de cada dia no começo dele. Os blocos estão numerados igual ao que o instrutor projeta, então é fácil se reencontrar.
- **Depois da aula:** o Apêndice, no fim de cada parte, é o que você mais vai usar. Comandos, asserções, mensagens de erro e o que cada uma significa. Ele cresce com o curso: cada parte traz tudo o que foi visto até aquele dia.

Todo o código desta apostila está no repositório do treinamento e **roda de verdade**. Nada aqui é pseudocódigo.

Antes do primeiro dia

Nada. **A instalação é feita em sala, no Bloco 1.3**, junto com o instrutor: Node.js, Git, VS Code, a loja que vamos testar e o Cypress. O passo a passo completo está em 1.3 — Instalação e configuração.

O treinamento é **online**. Você só precisa de:

- um computador com **Windows, Mac ou Linux** — a instalação traz os três — em que você possa **instalar programas**. Em computador de empresa, confirme antes que você tem essa permissão: é a causa número um de atraso no primeiro dia
- internet, e microfone (câmera é bem-vinda)
- de preferência, **duas telas** — ou uma tela grande —: numa você assiste, na outra você digita

Durante a aula, o chat é o seu canal: dúvidas, comandos para copiar e o ✅ de quem terminou cada passo. Se travar, o instrutor vai pedir que você compartilhe a sua tela.

A aplicação que vamos testar

O **ServeRest** é uma loja virtual de mentira, feita para servir de alvo em treinamentos de teste. Tem front-end (a loja) e API.

Cada aluno roda a sua cópia, no próprio computador, a partir do repositório serverest-treinamento:

Recurso	Endereço
Loja (front-end)	http://localhost:3001
API	http://localhost:3000

Usuário administrador do treinamento: `admin.treinamento@qa.com` / `treino123`

> DICA
> **Por que não a versão pública?** Existe uma, em `https://front.serverest.dev`, mas ela é compartilhada com o mundo inteiro: a base é apagada a cada poucos minutos, qualquer pessoa pode alterar o usuário que o seu teste usa, e ela bloqueia quando muita gente usa ao mesmo tempo. Tudo isso aconteceu durante a preparação deste treinamento — e vai virar conteúdo no Dia 3, quando falarmos de antipadrões.

Convenções desta apostila

> DICA
> **Vale lembrar** — algo que ajuda a fixar

> ATENÇÃO
> **Armadilha** — erro que quase todo mundo comete

CRÍTICO

Cuidado — algo que vai quebrar seu teste

JAVASCRIPT

```
// ❌ RUIM — o jeito que parece certo mas não é
// ✅ BOM — o jeito que funciona
```

Código em destaque aparece sempre completo o bastante para copiar e rodar.

Uma promessa sobre o terceiro dia

No fim do Dia 3 você vai ter escrito, do zero, uma suíte que:

- prepara os dados que precisa, sozinha, pela API
- entra na aplicação sem passar pela tela de login
- valida o que o usuário vê
- simula uma falha de servidor que você não conseguiria provocar na mão
- roda duas vezes seguidas sem quebrar
- e gera um relatório que outra pessoa entende sem você por perto

Se isso parece muito para três dias, é porque é. Mas nada disso é difícil isoladamente — e a gente vai um passo de cada vez.

Apostila do aluno · Treinamento Cypress · Material derivado do repositório `cypress-treinamento`, cujos exemplos foram executados e verificados contra o ServeRest real.

01

DIA 1

Fundamentos

AO FIM DESTE DIA VOCÊ TERÁ

um projeto Cypress rodando na sua máquina, uma suíte de testes escrita por você, e o hábito de ler a mensagem de erro em vez de adivinhar.

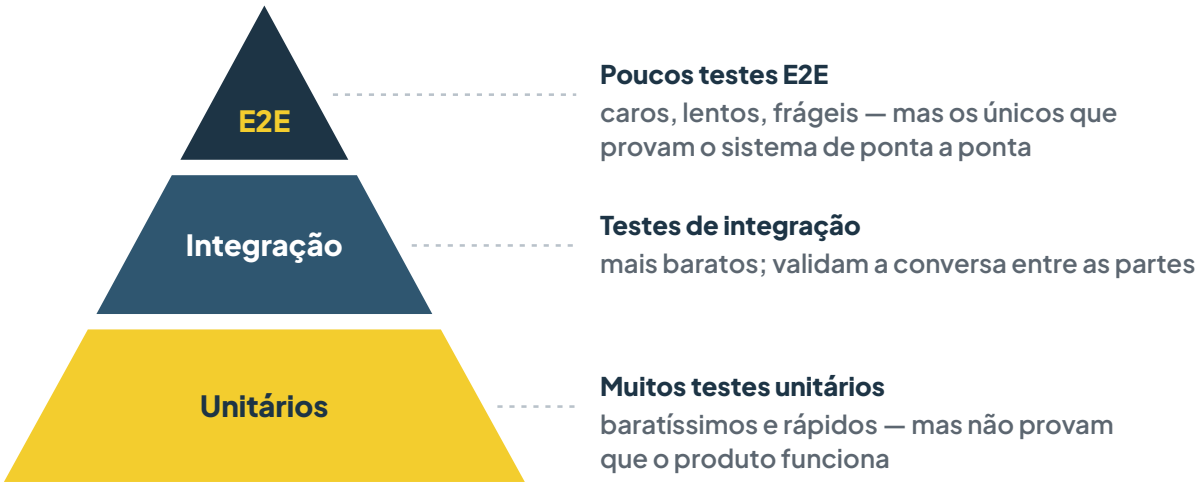
1.1 Estratégia antes da ferramenta

O que automatizar

Antes de aprender a escrever um teste automatizado, vale responder uma pergunta que quase ninguém faz: **quais testes vale a pena automatizar?**

A resposta que a maioria dos times descobre tarde demais: **não são todos.**

Teste automatizado tem dois custos. O de escrever, que é visível e todo mundo considera. E o de **manter**, que é invisível e é o que mata suítes. Cada teste que você escreve é um teste que alguém vai precisar consertar quando a tela mudar.



A pirâmide não é uma regra moral. É uma observação sobre custo: quanto mais alto, mais caro por teste e mais lento o retorno.

> DICA
>
> **A regra que vale a carreira inteira:** automatize o que **dói se quebrar**. Um erro no fluxo de pagamento dói. Um alinhamento torto no rodapé não dói o suficiente para justificar um teste E2E que alguém vai manter pelos próximos três anos.

O que normalmente vale automatizar em E2E:

- Fluxos que geram receita (compra, cadastro, checkout)
- Fluxos que, se quebrarem, param o negócio (login, autenticação)
- Caminhos que você já viu quebrar antes
- Cenários repetitivos e chatos de testar na mão

O que normalmente não vale:

- Tela que muda toda semana (o teste vai quebrar mais que a tela)
- Validação puramente visual (cor, fonte, espaçamento — existem ferramentas específicas para isso)
- Caso que acontece uma vez por ano
- Aquilo que um teste unitário já cobriria por um décimo do custo

O que é o Cypress

Cypress é uma ferramenta de teste automatizado para aplicações web. A característica que o define, e da qual decorre todo o resto:

> **O Cypress roda dentro do navegador, no mesmo processo que a sua aplicação.**

Ferramentas como o Selenium funcionam por fora: mandam comandos ao navegador por um protocolo e esperam a resposta, como alguém operando um computador por controle remoto. O Cypress está *dentro* da sala.

Consequências práticas disso:

O que você ganha	Por quê
Espera automática	O Cypress sabe quando a página mudou. Você não precisa mandar "espere 3 segundos"
Acesso ao que está por baixo	`window`, `localStorage`, requisições de rede — tudo acessível
Velocidade e estabilidade	Sem latência de protocolo entre a ferramenta e o navegador
Time Travel	Você volta no tempo e vê a tela em qualquer passo do teste

E o que você perde:

Limitação	O que significa na prática
Menos navegadores suportados	Chrome, Edge, Firefox e Electron. Sem Safari real
Múltiplas abas	Não dá para controlar duas abas ao mesmo tempo
Origem cruzada	Existem restrições ao navegar entre domínios diferentes

Origem cruzada, explicado. *Origem* é o trio **protocolo + domínio + porta**: `http://localhost:3001` e `http://localhost:3000` já são origens diferentes, só pela porta. O navegador isola uma origem da outra por segurança. Como o Cypress roda **dentro** da página, ele fica preso à origem que abriu. Se o fluxo sai para o site de outra empresa no meio do teste — um login pelo Google, um pagamento no site do banco —, o Cypress recusa. Desde a versão 12 existe o `cy.origin()` para esses casos, mas exige código a mais. É o preço de estar dentro da sala.

Quando não usar Cypress: teste de carga, aplicativo mobile nativo, ou scraping. Ferramenta errada para o trabalho.

> DICA
> **Scraping** é usar um robô para visitar sites e **extrair dados** deles: preços da concorrência, notícias, tabelas. O Cypress foi feito para testar uma aplicação, e fica preso a uma origem por vez. Para scraping existem ferramentas próprias, como o Puppeteer e o Playwright.

> DICA
> **"Cypress substitui o Selenium?"** Não. Resolve o mesmo problema com trade-offs diferentes. E existe o Playwright, que é concorrente direto e muito capaz. Desconfie de quem diz que só existe uma ferramenta boa.

1.2 JavaScript essencial

O enquadramento honesto

Você **não** precisa virar desenvolvedor. Precisa ler e escrever cinco construções. É menos do que parece.

E o que você **não** vai precisar neste treinamento: herança, `async/await`, módulos avançados, TypeScript. Se você já ouviu essas palavras e se assustou, pode relaxar.

As cinco construções

1. `const` e `let` — guardar um valor com nome

JAVASCRIPT

```
const url = 'http://localhost:3001'        // não vai mudar
let contador = 0                           // vai mudar
```

Use `const` por padrão. Use `let` só quando o valor realmente for trocar.

2. Template literal — montar texto com variável dentro

```javascript
const email = `usuario_${Date.now()}@qa.com`
```

Repare no acento agudo invertido (`` ` ``), não na aspa. O `${...}` é o buraco onde entra o valor. `Date.now()` devolve o número de milissegundos desde 1970 — ou seja, **um número diferente a cada vez**. É assim que a gente vai garantir e-mails únicos.

3. Objeto — agrupar dados que andam juntos

```javascript
const usuario = {
  nome: 'Fulano',
  email: 'fulano@exemplo.com',
  admin: true
}

usuario.nome        // 'Fulano'
usuario['email']    // 'fulano@exemplo.com'
```

4. Array e `forEach` — uma lista e o que fazer com cada item

```javascript
const cenarios = ['válido', 'inválido', 'vazio']

cenarios.forEach((cenario) => {
  console.log(cenario)
})
```

Leia assim: *"para cada item da lista, que eu vou chamar de `cenario`, faça isto"*. No Dia 2 é exatamente isso que vai gerar um teste por linha de uma planilha de dados.

5. Arrow function — uma função escrita curto

```javascript
const somar = (a, b) => a + b
```

É a mesma coisa que `function somar(a, b) { return a + b }`, escrita em menos caracteres. Você vai ver `() => { ... }` o tempo todo no Cypress.

Callback — o conceito que mais confunde

Um **callback** é uma função que você entrega para outra pessoa executar *depois*, quando ela terminar o trabalho dela.

JAVASCRIPT

```javascript
cy.get('[data-testid="email"]').then((elemento) => {
  // este código só roda DEPOIS que o elemento foi encontrado
})
```

DICA

A analogia do café. Você pede um café e diz ao barista: "quando ficar pronto, me chama". Você não fica parado olhando a máquina. O `.then()` é o "me chama".

É por isso que o valor só existe **dentro** do `.then()`. Fora dele, o café ainda não ficou pronto.

Desestruturação e spread

Vão aparecer no material. Não precisa decorar agora, só reconhecer:

JAVASCRIPT

```javascript
const { nome, email } = usuario          // tira nome e email do objeto
const novoUsuario = { ...usuario, admin: false }  // copia tudo e troca uma coisa
```

ATENÇÃO

Semente para o Dia 2. Existe uma diferença entre `() => {}` e `function () {}` que vai importar amanhã, no bloco de alias. Por enquanto, guarde só que a diferença existe. Explicar agora, sem um problema real na frente, não fixaria.

Exercício de leitura (3 min em sala): diga **em português** o que cada trecho faz. Não precisa escrever código. Em sala, o instrutor pede o **a** e o **d**; os outros três ficam para você.

JAVASCRIPT

```javascript
// a
const email = `usuario_${Date.now()}@qa.com`

// b
const cenarios = ['válido', 'inválido', 'vazio']
cenarios.forEach((cenario) => { console.log(cenario) })

// c
const novoUsuario = { ...usuario, admin: false }

// d
cy.get('[data-testid="email"]').then((elemento) => { console.log(elemento) })

// e
const { nome, email } = usuario
```

Os trechos são para ler, um de cada vez; não rode o arquivo inteiro. Considere que `usuario` é o objeto da construção 3.

Respostas — confira depois de tentar:

	O que o trecho faz
a	Monta um e-mail diferente a cada execução: `usuario_`, mais o número de milissegundos de agora, mais `@qa.com`. Por exemplo, `usuario_1790000000000@qa.com`
b	Cria uma lista com três textos e, para cada um, escreve o texto no console. Resultado: três linhas
c	Cria um objeto novo, cópia de `usuario`, com `admin` trocado para `false`. O `usuario` original não muda
d	Procura o campo de e-mail e, **quando achar**, entrega o elemento para a função, que o imprime no console. Se você respondeu *"guarda o elemento na variável `elemento`"*, releia a analogia do café: nada é guardado, a função é chamada depois
e	Tira `nome` e `email` de dentro de `usuario` e cria duas variáveis com esses nomes. É o mesmo que `const nome = usuario.nome` e `const email = usuario.email`

1.3 Instalação e configuração

Tudo o que você precisa instalar está aqui, na ordem. São cinco passos; siga junto com o instrutor. Se já tiver algum programa instalado, confira a versão e pule para o próximo.

Passo	O quê	Para quê
1	Node.js	É o que roda o Cypress e a loja
2	Git	Baixa a loja do repositório
3	VS Code	Onde você escreve os testes
4	A loja na sua máquina	A aplicação que vamos testar
5	O projeto Cypress	A sua suíte de testes

Cada passo tem instruções para Windows, Mac e Linux. Siga só as do seu sistema. Onde não há separação, é igual nos três.

Antes de começar: o terminal

Quase tudo aqui é digitado num **terminal** — uma janela onde você escreve comandos. Para abrir um:

Sistema	Como abrir
Windows	Tecla Windows, digite `PowerShell`, Enter
Mac	Cmd + Espaço, digite `Terminal`, Enter
Linux	Ctrl + Alt + T (Ubuntu e a maioria das distribuições)

> **ATENÇÃO**
> **Depois de instalar qualquer programa, feche o terminal e abra de novo.** Um terminal aberto antes da instalação não enxerga o programa novo, e responde "não é reconhecido" ou "command not found".

Passo 1 — Node.js

Windows e Mac

1. Abra nodejs.org e baixe a versão **LTS** — é a estável, a que o site oferece em destaque. O site já detecta o seu sistema: no Windows vem um `.msi`, no Mac um `.pkg`
2. Rode o instalador e aceite tudo como está até o fim
3. Feche e abra o terminal

Linux

Não use o `nodejs` do gerenciador de pacotes da distribuição: costuma ser uma versão antiga demais para o Cypress. Instale pelo **nvm**, que é o que o próprio nodejs.org recomenda para Linux:

TERMINAL

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
```

Feche e abra o terminal. Depois:

TERMINAL

```
nvm install --lts
```

> **DICA**
> Se o `curl` não existir: `sudo apt install curl` (Ubuntu/Debian) ou `sudo dnf install curl` (Fedora). Se o nodejs.org mostrar outra versão do nvm no comando, use a de lá.

Nos três sistemas, confira:

TERMINAL

```
node -v
npm -v
```

Os dois precisam responder com um número. O Node precisa ser **20 ou mais novo** (`v20...`, `v22...`, `v24...`).

ATENÇÃO

Só Windows — "a execução de scripts foi desabilitada neste sistema". Se o `npm -v` responder isso no PowerShell, é uma trava de segurança do Windows. Rode uma vez, responda `S` (ou `Y`) e tente de novo:

POWERSHELL

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Passo 2 — Git

Windows

1. Abra git-scm.com e baixe o instalador
2. Rode e aceite tudo como está. São muitas telas; o padrão de todas serve
3. Feche e abra o terminal

Mac

No Terminal, rode `git --version`. Se o Git não estiver instalado, o próprio macOS oferece instalar as **ferramentas de linha de comando** (*command line developer tools*): clique em **Instalar** e espere — são alguns minutos. Depois, feche e abra o Terminal.

Linux

TERMINAL

```
sudo apt install git        # Ubuntu, Debian, Mint
sudo dnf install git        # Fedora
```

Nos três sistemas, confira:

TERMINAL

```
git --version
```

Passo 3 — VS Code

ATENÇÃO

Visual Studio Code, não Visual Studio. São programas diferentes, da mesma empresa. O Visual Studio (sem "Code") é enorme, pago em algumas versões, e não é o que usamos. O ícone do certo é azul.

Baixe em code.visualstudio.com. O site oferece o arquivo do seu sistema.

Sistema	Como instalar
Windows	Rode o instalador aceitando o padrão. Se aparecer **"Adicionar ao PATH"**, deixe marcado
Mac	Abra o `.zip` baixado e arraste o **Visual Studio Code** para a pasta **Aplicativos**
Linux (Ubuntu/Debian)	Baixe o `.deb` e, na pasta de downloads: `sudo apt install ./code_*.deb`
Linux (Fedora)	Baixe o `.rpm` e, na pasta de downloads: `sudo dnf install ./code-*.rpm`

Ainda não abra nada nele: o VS Code entra no Passo 5.

Passo 4 — A loja na sua máquina

O ServeRest tem uma versão pública na internet, mas ela é compartilhada com o mundo inteiro: a base é apagada a cada poucos minutos e bloqueia quando muita gente usa ao mesmo tempo — que é exatamente o que uma turma faz. Por isso cada um roda a sua cópia.

Abra um terminal (o do sistema, não o do VS Code) e rode, uma linha de cada vez.

Windows (PowerShell):

POWERSHELL

```
mkdir C:\treinamento
cd C:\treinamento
git clone https://github.com/jhonatangoncalvesYwp/serverest-treinamento.git
cd serverest-treinamento
npm install
npm start
```

Mac e Linux (Terminal):

TERMINAL

```
mkdir ~/treinamento
cd ~/treinamento
git clone https://github.com/jhonatangoncalvesYwp/serverest-treinamento.git
cd serverest-treinamento
npm install
npm start
```

> **DICA**
> O `~` é a sua pasta pessoal (`/Users/seu-nome` no Mac, `/home/seu-nome` no Linux). Daqui em diante, onde a apostila disser `C:\treinamento` , no Mac e no Linux leia `~/treinamento` .

Quando aparecer:

```
ServeRest local pronto

Loja:  http://localhost:3001
API:   http://localhost:3000
Admin: admin.treinamento@qa.com / treino123
```

abra **http://localhost:3001** no navegador. A tela de login da loja precisa aparecer. Entre com o admin para conferir: você cai numa tela que diz **Bem Vindo Administrador do Treinamento**.

> **CRÍTICO**
> **Esta janela fica aberta o dia inteiro.** Ela é a loja. Minimize, mas não feche: se ela fechar, todos os testes falham. E rode a loja nesta janela, não no terminal do VS Code — abrir outra pasta no VS Code fecha os terminais dele, e a loja iria junto.

Nos dias seguintes, é só abrir o terminal de novo e rodar:

TERMINAL

```
cd C:\treinamento\serverest-treinamento     # Windows
cd ~/treinamento/serverest-treinamento      # Mac e Linux
npm start
```

Os dados que você cadastrou continuam lá.

Passo 5 — O projeto Cypress

1. Abra o VS Code
2. **File → Open Folder** (Arquivo → Abrir Pasta). No Mac o menu pode se chamar **File → Open…** — funciona igual
3. Entre em `C:\treinamento` (Mac e Linux: `treinamento`, na sua pasta pessoal), crie uma pasta nova chamada `cypress-treinamento` (botão **Nova pasta** da janela) e selecione-a
4. Se o VS Code perguntar se você confia nos autores da pasta, responda **Yes, I trust the authors**

Agora abra o terminal **do VS Code**: menu **Terminal → New Terminal**. Ele já abre dentro de `cypress-treinamento`. Rode:

TERMINAL

```
npm init -y
npm install cypress@15.21.1 --save-dev
```

O `@15.21.1` fixa a versão: é a mesma do instrutor e a versão em que todo o material foi testado. Sem ele, o npm instala a mais nova — o Cypress 16, que saiu em setembro de 2026 e removeu o `Cypress.env()`, usado neste material a partir do Dia 2. Numa suíte de verdade, fixar a versão também é boa prática: todo mundo do time roda igual.

O `npm install` baixa o Cypress inteiro (algumas centenas de MB) e pode levar alguns minutos. Quando terminar:

TERMINAL

```
npx cypress open
```

Na primeira vez, o Cypress confere a própria instalação antes de abrir — leva um minuto. Na janela que aparecer, escolha **E2E Testing**. O Cypress cria a estrutura de pastas sozinho.

ATENÇÃO

Só Linux — se o Cypress não abrir e reclamar de biblioteca. O Cypress usa algumas bibliotecas gráficas do sistema. Distribuições desktop quase sempre já têm. Se faltar alguma, o Cypress avisa ao abrir: a mensagem começa com `Your system is missing the dependency` ou cita um arquivo `.so` que não foi encontrado. No Ubuntu 24.04 ou mais novo:

TERMINAL

```
sudo apt install libgtk2.0-0t64 libgtk-3-0t64 libgbm-dev libnotify-dev libnss3 libxss1
libasound2t64 libxtst6 xauth xvfb
```

Em outras versões e distribuições, a lista certa está em docs.cypress.io — Linux Prerequisites.

Ao fim do bloco, você tem duas pastas e duas janelas:

```
treinamento/
├── serverest-treinamento/   ← a loja: janela de terminal, sempre aberta
└── cypress-treinamento/     ← os seus testes: aberta no VS Code
```

O arquivo de configuração

JAVASCRIPT

```
const { defineConfig } = require('cypress')

module.exports = defineConfig({
  e2e: {
    baseUrl: 'http://localhost:3001',
    viewportWidth: 1280,
    viewportHeight: 720,
    defaultCommandTimeout: 10000,
    video: true,
    screenshotOnRunFailure: true,
    env: {
      apiUrl: 'http://localhost:3000'
    },
    pageLoadTimeout: 30000,
    setupNodeEvents(on, config) {
      return config
    }
  }
})
```

As duas URLs são a loja e a API que o `npm start` do Passo 4 subiu na sua máquina.

Linha por linha:

Opção	O que faz
`baseUrl`	Endereço base. Permite escrever `cy.visit('/login')` em vez da URL inteira
`viewportWidth` / `Height`	Tamanho da janela do navegador durante o teste
`defaultCommandTimeout`	Quanto tempo o Cypress insiste antes de desistir. Padrão é 4s; aqui, 10s
`video`	Grava vídeo da execução (só no modo headless). Vem desligado por padrão desde o Cypress 13
`screenshotOnRunFailure`	Tira print automático quando um teste falha
`env`	Valores seus. Aqui guardamos a URL da API
`pageLoadTimeout`	Quanto tempo esperar uma página abrir. O padrão é 60s; 30s dá folga sem esconder uma loja fora do ar
`setupNodeEvents`	Onde se ligam plugins. Hoje está vazio; no Dia 3 ele recebe o relatório e o Cucumber

DICA

`baseUrl` é mais importante do que parece. Além de encurtar o `cy.visit()`, é o que permite rodar a mesma suíte em outro ambiente trocando **uma linha**. Homologação, produção, ambiente local — mesmo teste. Se um dia você quiser rodar contra o ServeRest público, é trocar as duas URLs por `https://front.serverest.dev` e `https://serverest.dev`.

Scripts no `package.json`

JSON

```
{
  "scripts": {
    "cy:open": "cypress open",
    "cy:run": "cypress run",
    "cy:run:chrome": "cypress run --browser chrome"
  }
}
```

Agora você roda `npm run cy:open` em vez de lembrar o comando inteiro.

`.gitignore`

```
node_modules/
cypress/videos/
cypress/screenshots/
cypress/downloads/
```

Nada disso vai para o repositório: são coisas geradas, e pesam.

Se a instalação travar

É o momento mais provável de dar problema. Causas comuns, em ordem:

Sintoma	Causa provável
`node`, `npm` ou `git` "não é reconhecido como comando" / "command not found"	O terminal foi aberto antes da instalação. Feche e abra de novo
`nvm: command not found` (Linux)	Mesmo caso: feche e abra o terminal depois de instalar o nvm
"A execução de scripts foi desabilitada neste sistema" (Windows)	Trava do PowerShell. Veja o Passo 1
`Your system is missing the dependency`, ou um arquivo `.so` (Linux)	Falta biblioteca do sistema. Veja o fim do Passo 5
`A porta 3000 (API) já está em uso`	Já existe um `npm start` da loja aberto em outra janela. Use aquele
A loja abre, mas o login não faz nada	A janela da loja foi fechada. Rode o `npm start` do Passo 4 de novo
`Cypress could not verify that this server is running` ou `cy.visit() failed trying to load`	Mesmo caso: a loja não está no ar
Download do Cypress para no meio	Proxy da empresa bloqueando
`EACCES` / permissão negada	Instalação do Node com permissões erradas
O executável não abre	Antivírus barrando o binário
`bad option: --smoke-test`	Variável `ELECTRON_RUN_AS_NODE` no terminal (comum no VS Code)
`Cypress.env() was removed in Cypress version 16.0.0`	Instalou sem o `@15.21.1`. Rode `npm install cypress@15.21.1 --save-dev` na pasta do projeto

Esse último tem solução rápida. Antes de rodar:

TERMINAL

```
unset ELECTRON_RUN_AS_NODE          # Git Bash / Linux / Mac
```

POWERSHELL

```
$env:ELECTRON_RUN_AS_NODE=$null     # PowerShell
```

DICA

Se você travar: avise no chat e compartilhe a sua tela quando o instrutor pedir. Mostrar a tela resolve em cinco minutos o que descrever o problema por escrito leva meia hora — e você não perde o resto do bloco.

EXERCÍCIO · 5 MIN

troque `viewportWidth` para `1440`, salve, reabra o Cypress e observe. Depois volte para `1280`. O objetivo é criar o hábito de mexer na configuração sem medo.

1.4 Seu primeiro teste

Anatomia

JAVASCRIPT

```
describe('Página de login', () => {      // agrupa testes relacionados
  it('deve carregar a tela de login', () => {   // um teste
    cy.visit('/login')                                  // comando
    cy.get('[data-testid="email"]').should('be.visible')   // comando + asserção
  })
})
```

Quatro peças, e é só isso:

- `describe` — a pasta. Agrupa testes que falam do mesmo assunto
- `it` — o teste. Um cenário, um resultado esperado
- **comando** — `cy.alguma_coisa()`. Faz algo na página
- **asserção** — `.should(...)`. Afirma que algo é verdade

De onde vem essa sintaxe

O Cypress **não inventou** `describe`, `it` nem `expect`. Ele empacota bibliotecas que já eram padrão no mundo JavaScript:

Vem de	O que você usa dele
Mocha	O executor de testes: `describe`, `it`, `before`, `beforeEach`, `after`, `afterEach`, `it.only`, `it.skip`
Chai	As asserções: `expect`, e o vocabulário `to.eq`, `to.contain`, `to.be.an`
jQuery	O motor de seleção por trás do `cy.get()`, e o objeto que chega no `.then()`

O que é cada um, em uma frase:

- **Mocha** é quem **organiza e executa** os testes. Ele dá a estrutura — `describe` para agrupar, `it` para cada teste, os hooks do bloco 1.6 — e, no fim, diz o que passou e o que falhou. Pense nele como o roteiro da prova.

- **Chai** é o **vocabulário da verificação**. `expect(total).to.eq(3)` se lê quase como uma frase em inglês: *"espero que o total seja igual a 3"*. É com ele que o teste decide se algo está certo.
- **jQuery** é uma biblioteca antiga e muito conhecida para **achar e mexer em elementos** de uma página, famosa pelo `$('seletor')`. O `cy.get()` usa o jQuery por baixo para encontrar os elementos. Por isso o que chega dentro de um `.then()` é um *objeto jQuery*: se você imprimir no console, vai ver esse embrulho em volta do elemento — como no trecho **d** do exercício do 1.2.

> DICA
> **Por que você precisa saber disso.** Quando você pesquisar `describe javascript` no Google, vai cair na documentação do **Mocha** — não do Cypress — e pode achar que está no lugar errado. Não está. Saber o nome das peças é o que permite pesquisar direito depois do treinamento.

O que o Cypress acrescentou por cima: o `.should()` com nova tentativa automática, que o Chai puro não tem. É a diferença entre:

JAVASCRIPT

```
expect(valor).to.eq('x')                    // Chai: roda uma vez
cy.get('.alerta').should('contain', 'x')    // Cypress: tenta de novo até passar
```

Guarde essa diferença — ela volta no bloco 1.5.

> DICA
> **O nome do teste é parte do teste.** `it('deve carregar a tela de login')` lê-se como uma frase: *"deve carregar a tela de login"*. Quando esse teste falhar no relatório, essa frase é tudo que a pessoa vai ver. Escreva pensando nela.

O primeiro teste

Este é o teste que o instrutor escreve em sala, no arquivo `cypress/e2e/dia1/01-primeiro-teste.cy.js`:

JAVASCRIPT

```
describe('Página de login', () => {
  it('deve carregar a tela de login', () => {
    cy.visit('/login')

    cy.get('[data-testid="email"]').should('be.visible')
    cy.get('[data-testid="senha"]').should('be.visible')
    cy.contains('button', 'Entrar').should('be.visible')
  })
})
```

Três asserções: o campo de e-mail, o de senha e o botão **Entrar** estão na tela.

No Cypress, passe o mouse devagar sobre os comandos da lista à esquerda: a página muda para o exato instante daquele comando. Isso é o **Time Travel** prometido no bloco 1.1 — não é um print, é o estado da página naquele passo.

Aprendendo a errar de propósito

Troque o seletor por `[data-testid="emai"]` — sem o "l" — e rode.

Você vai ver o Cypress insistir por 10 segundos e então falhar com:

```
Timed out retrying after 10000ms:
Expected to find element: [data-testid="emai"], but never found it.
```

Leia essa mensagem com atenção. Ela diz exatamente o que o Cypress procurou e que não achou. Noventa por cento dos seus erros nos próximos meses vão ser variações disso.

> DICA
>
> **Ler o erro vale mais que decorar o comando.** Você vai esquecer a sintaxe de `cy.select()` e consultar a documentação. Mas se você não souber ler uma falha, vai travar toda vez.

Volte para `email` e rode de novo: verde.

Login que funciona

JAVASCRIPT

```javascript
it('deve permitir login com credenciais válidas de administrador', () => {
  cy.visit('/login')
  cy.get('[data-testid="email"]').type('admin.treinamento@qa.com')
  cy.get('[data-testid="senha"]').type('treino123')
  cy.get('[data-testid="entrar"]').click()

  cy.url().should('include', '/admin/home')
  cy.contains('h1', 'Bem Vindo').should('be.visible')
})
```

> ATENÇÃO
>
> **Cuidado com o que você valida depois do login.** O ServeRest tem duas telas iniciais: a do administrador (`/admin/home` , que mostra `Bem Vindo <nome>`) e a da loja (`/home` , que mostra `Serverest Store`). Como `admin.treinamento@qa.com` é administrador, ele cai na primeira. Validar `Serverest Store` aqui faria o teste falhar — e é um erro fácil de cometer copiando exemplo da internet.

As duas formas de rodar

Modo interativo — `npx cypress open`

É onde você escreve. Tem Time Travel (passe o mouse sobre cada comando e veja a tela naquele instante), recarregamento automático ao salvar, e o seletor de elementos (o ícone de mira).

Modo headless — `npx cypress run`

É onde o teste roda "para valer": sem janela, mais rápido, com vídeo e print das falhas.

TERMINAL

```
npx cypress run
npx cypress run --spec "cypress/e2e/dia1/01-primeiro-teste.cy.js"
npx cypress run --headed
```

DICA

Um aviso amarelo sobre `allowCypressEnv` aparece em todo `npx cypress run`. É esperado: o Cypress 15 avisa que o `Cypress.env()`, que este material usa, vai ser removido numa versão futura. Hoje nada quebra. A explicação completa está no bloco 2.5.

Escolhendo o navegador

TERMINAL

```
npx cypress run --browser chrome
npx cypress run --browser edge
npx cypress run --browser electron    # o padrão, embutido no Cypress
```

No modo interativo, a lista de navegadores fica no canto superior direito — você troca com um clique e roda de novo.

Três coisas que vale saber:

- **Electron é o padrão** e vem embutido no Cypress. É por isso que o teste roda mesmo numa máquina sem Chrome instalado
- O Cypress só enxerga navegadores **instalados na máquina**. No servidor de integração, quem resolve isso é a imagem Docker — você vai ver no Dia 3
- **Safari não é suportado oficialmente.** Existe um suporte experimental ao WebKit, o motor do Safari, mas não é o Safari de verdade. Se o seu time precisa cobrir Safari, esse é um argumento legítimo a favor de outra ferramenta, como o Playwright

DICA

Aqui aquela limitação da seção 1.1 deixa de ser item de lista e vira algo que você vê na sua própria tela.

DICA

Guarde este comando. `npx cypress run` é *exatamente* o mesmo comando que vai rodar no servidor de integração contínua, no Dia 3. Não existe mágica no CI — existe outra máquina rodando o que você já roda aqui.

EXERCÍCIO · 5 MIN

escreva um teste que valide a mensagem de erro ao tentar logar com a **senha errada**. Use o "Login que funciona" como ponto de partida: o que muda, e o que continua igual? A mensagem que a loja mostra é `Email e/ou senha inválidos`.

Resolução: login que falha

Confira depois de tentar.

JAVASCRIPT

```
it('deve exibir mensagem de erro ao informar senha inválida', () => {
  cy.visit('/login')
  cy.get('[data-testid="email"]').type('admin.treinamento@qa.com')
  cy.get('[data-testid="senha"]').type('senhaerrada')
  cy.get('[data-testid="entrar"]').click()

  cy.get('.alert').should('contain', 'Email e/ou senha inválidos')
  cy.url().should('include', '/login')
})
```

Repare na segunda asserção: não basta a mensagem aparecer, o usuário também tem que **continuar** na tela de login. Testar só metade do comportamento é um jeito silencioso de deixar bug passar.

DICA

Para praticar em casa: faça o mesmo com um e-mail que não está cadastrado. A mensagem é a mesma — de propósito: a loja não conta a quem tenta invadir se o e-mail existe.

1.5 Comandos e asserções

Comandos por finalidade

Navegar

JAVASCRIPT

```
cy.visit('/login')       // abre uma página
cy.reload()              // recarrega
cy.go('back')            // volta
cy.url()                 // a URL atual
cy.title()               // o título da aba
```

Encontrar

```javascript
cy.get('[data-testid="email"]')   // por seletor CSS
cy.contains('Entrar')             // por texto visível
cy.get('form').find('input')      // dentro do elemento anterior
cy.get('li').first()              // o primeiro
cy.get('li').last()               // o último
cy.get('li').eq(2)                // o terceiro (contando do zero)
```

Interagir

```javascript
.click()
.type('texto')
.clear()
.check()        // marca checkbox ou radio
.uncheck()
.select('opção')   // para <select>
```

Afirmar

```javascript
.should('be.visible')
.and('contain', 'texto')   // .and é um .should encadeado
```

Encadeamento

Os comandos se encaixam um no outro. Cada um recebe o resultado do anterior:

```javascript
cy.get('.lista-produtos')      // acha o elemento
  .find('li')                  // acha os filhos dele
  .should('have.length', 3)    // afirma sobre os filhos
```

O erro que todo iniciante comete

```javascript
// ❌ NÃO FUNCIONA
cy.get('[data-testid="email"]').type('admin.treinamento@qa.com')

const email = cy.get('[data-testid="email"]').invoke('val')
expect(email).to.eq('admin.treinamento@qa.com')
```

Parece razoável: o campo tem exatamente esse texto. Mas o teste falha com:

```
expected { Object (specWindow, chainerId) } to equal 'admin.treinamento@qa.com'
```

Comandos do Cypress não devolvem valores — eles entram numa fila para serem executados depois. Na hora em que a linha do `const` roda, o elemento nem foi procurado ainda. O que `email` guarda é um objeto interno do Cypress, não o texto.

JAVASCRIPT

```
// ✅ Se você quer afirmar algo, afirme direto
cy.get('[data-testid="email"]').should('have.value', 'admin.treinamento@qa.com')

// ✅ Se você precisa mesmo do valor, use o callback
cy.get('[data-testid="email"]')
  .invoke('val')
  .then((valor) => {
    cy.log(valor)
    expect(valor).to.eq('admin.treinamento@qa.com')
  })
```

DICA

É o café do bloco 1.2 de novo. O valor não existe na hora em que você escreve a linha. Ele existe quando ficar pronto — e o `.then()` é onde ele chega.

`should` ou `expect` ?

	Quando usar
`.should()`	Na maioria das vezes. **Tenta de novo** até passar ou estourar o tempo
`expect()`	Dentro de um `.then()`, quando o valor já está em mãos. Roda **uma vez só**

Usar `expect()` onde caberia `.should()` é uma das causas mais comuns de teste instável: você afirma antes de a tela terminar de reagir.

Retry-ability: o motivo de não existir `cy.wait(3000)`

JAVASCRIPT

```
// ❌ ANTIPADRÃO
cy.wait(3000)
cy.get('.mensagem').should('be.visible')

// ✅ CORRETO — o Cypress tenta de novo sozinho até os 10 segundos
cy.get('.mensagem').should('be.visible')

// ✅ CORRETO para um caso específico que demora mais
cy.get('.relatorio', { timeout: 20000 }).should('be.visible')
```

Por que `cy.wait(3000)` é ruim mesmo quando funciona:

- Se a tela responde em 200ms, você desperdiçou 2,8 segundos. **Vezes trinta testes.**

- Se num dia ruim ela demora 3,5s, seu teste falha sem motivo real.

O número certo não existe. O `.should()` resolve os dois casos: ele volta e tenta de novo até dar certo ou até o tempo acabar.

> **DICA**
> A única espera legítima é por um **acontecimento**, não por um tempo. Você vai ver isso amanhã, no bloco de alias.

EXERCÍCIO · 20 MIN

escreva três testes para a tela `/cadastrarusuarios`: cadastro válido com e-mail único, e-mail já existente, campo obrigatório vazio.

> **ATENÇÃO**
> Atenção ao campo de senha nessa tela: ele é `[data-testid="password"]`, não `senha`. `senha` é só do login. Se você copiar da tela anterior, vai ter um bom erro para ler.

> **CRÍTICO**
> **No cenário de e-mail existente, não use o e-mail do admin.** Cadastre um e-mail novo e tente cadastrá-lo de novo. Com o do admin, o teste passa na sua máquina, mas passa a depender de o admin existir. Num ambiente compartilhado, como o ServeRest público, o admin some de tempos em tempos — e aí o seu teste o recadastraria como usuário comum, quebrando o login de todo mundo. Aconteceu durante a preparação deste treinamento.

1.6 Hooks e independência de testes

O que é um hook

Hook é "gancho" em inglês. É uma função que **você não chama**: você pendura o código num gancho, e o Mocha (lembra do bloco 1.4?) o executa sozinho, num momento fixo da vida dos testes — antes de todos, antes de cada um, depois de cada um, no fim.

Pense no checklist do piloto antes de **cada** decolagem. Ninguém precisa lembrar de fazer: faz parte do procedimento. O `beforeEach` é esse checklist.

Serve para tirar de dentro dos testes o que se repete no começo de todos eles — abrir a mesma tela, gerar um e-mail novo — e escrever uma vez só.

Os quatro hooks

JAVASCRIPT

```javascript
describe('Cadastro de usuário', () => {
  let emailUnico

  before(() => {
    cy.log('roda uma vez, antes de todos os testes')
  })

  beforeEach(() => {
    emailUnico = `usuario_${Date.now()}@qa.com`   // um e-mail novo por teste
    cy.visit('/cadastrarusuarios')                 // antes de CADA teste
  })

  afterEach(() => {
    cy.log('roda depois de cada teste')
  })

  after(() => {
    cy.log('roda uma vez, no final')
  })

  it('deve cadastrar um usuário comum', () => {
    cy.get('[data-testid="nome"]').type('Usuário Comum')
    cy.get('[data-testid="email"]').type(emailUnico)
    cy.get('[data-testid="password"]').type('teste123')
    cy.get('[data-testid="cadastrar"]').click()

    cy.get('.alert').should('contain', 'Cadastro realizado com sucesso')
  })

  it('deve cadastrar um usuário administrador', () => {
    cy.get('[data-testid="nome"]').type('Usuário Administrador')
    cy.get('[data-testid="email"]').type(emailUnico)
    cy.get('[data-testid="password"]').type('teste123')
    cy.get('[data-testid="checkbox"]').check()
    cy.get('[data-testid="cadastrar"]').click()

    cy.get('.alert').should('contain', 'Cadastro realizado com sucesso')
    cy.url().should('include', '/admin/home')
  })
})
```

Hook	Quando roda
`before`	Uma vez, antes do primeiro teste do `describe`
`beforeEach`	Antes de **cada** teste
`afterEach`	Depois de cada teste
`after`	Uma vez, depois do último teste

Na prática, `beforeEach` **é o que você vai usar 90% das vezes**: é onde mora tudo que se repete no começo dos testes. Repare que nenhum dos dois testes abre a tela nem inventa e-mail: o `beforeEach` já fez isso por eles.

> DICA
>
> **Por que o e-mail nasce no `beforeEach`, e não no topo do arquivo?** No topo, o `Date.now()` rodaria uma vez só, e os dois testes cadastrariam o mesmo e-mail — o segundo receberia "Este email já está sendo usado". No `beforeEach`, cada teste ganha o seu.

A regra

> **Cada `it` tem que rodar sozinho, em qualquer ordem, quantas vezes for.**

Não é preciosismo. É o que separa uma suíte útil de uma suíte que ninguém confia.

Por que isso importa tanto

Imagine dois testes onde o segundo depende do primeiro:

JAVASCRIPT

```
let email

it('deve cadastrar um usuário', () => {
  email = `dor_${Date.now()}@qa.com`
  cy.visit('/cadastrarusuarios')
  cy.get('[data-testid="nome"]').type('Usuário da Dor')
  cy.get('[data-testid="email"]').type(email)
  cy.get('[data-testid="password"]').type('teste123')
  cy.get('[data-testid="cadastrar"]').click()
  cy.get('.alert').should('contain', 'Cadastro realizado com sucesso')
})

it('deve fazer login com o usuário cadastrado', () => {
  cy.visit('/login')
  cy.get('[data-testid="email"]').type(email)   // e o email? veio do teste anterior
  cy.get('[data-testid="senha"]').type('teste123')
  cy.get('[data-testid="entrar"]').click()
  cy.contains('h1', 'Serverest Store').should('be.visible')
})
```

Rode os dois: passam. Agora rode **só o segundo** (troque `it` por `it.only`): falha com `cy.type() can only accept a string or number. You passed in: undefined`.

O segundo teste nunca soube de onde vinha o e-mail. Ele estava pegando carona.

Isso quebra no dia a dia: quando alguém roda só aquele teste para investigar um bug, ou quando o primeiro falha e derruba o segundo junto. E aí o relatório acusa o teste errado — você vai investigar "fazer login", que está perfeito, quando o problema é que ninguém cadastrou o usuário.

DICA

O Cypress já te protege de metade disso. Desde a versão 12, ele limpa cookies e `localStorage` entre os testes (a `testIsolation`). Por isso um teste que tenta "herdar o login" do anterior falha na hora. O que ele não limpa é o que está fora do navegador: variáveis do arquivo e dados no servidor. Essa parte é com você.

A versão independente

O teste de login passa a criar o próprio usuário antes de entrar. Não depende de ninguém:

JAVASCRIPT

```
describe('Testes independentes', () => {
  it('deve fazer login com um usuário criado pelo próprio teste', () => {
    const email = `independente_${Date.now()}@qa.com`

    cy.visit('/cadastrarusuarios')
    cy.get('[data-testid="nome"]').type('Usuário Independente')
    cy.get('[data-testid="email"]').type(email)
    cy.get('[data-testid="password"]').type('teste123')
    cy.get('[data-testid="cadastrar"]').click()
    cy.get('.alert').should('contain', 'Cadastro realizado com sucesso')

    cy.visit('/login')
    cy.get('[data-testid="email"]').type(email)
    cy.get('[data-testid="senha"]').type('teste123')
    cy.get('[data-testid="entrar"]').click()
    cy.contains('h1', 'Serverest Store').should('be.visible')
  })
})
```

Rode com `it.only`, rode o arquivo inteiro, rode duas vezes: passa sempre.

Sim, ficou mais longo — o cadastro pela tela custa tempo. No Dia 2 você vai aprender a preparar esse usuário **pela API**, em milissegundos, e o teste volta a ter só o que importa: o login.

DICA

O teste do teste: rode sua suíte **duas vezes seguidas**. Se a segunda falhar, algum teste está dependendo de estado deixado por outro — ou pelo próprio teste na rodada anterior.

Para praticar em casa (15 min): pegue os três testes do seu exercício do bloco 1.5 e extraia para `beforeEach` tudo que se repete — como no exemplo de **Os quatro hooks**, que é essa mesma refatoração feita em sala. Depois rode cada teste isoladamente com `it.only` para provar que continuam funcionando sozinhos.

1.7 Seletores e como se desbloquear sozinho

A hierarquia dos seletores

Do melhor para o pior:

	Tipo	Exemplo	Veredito
1	Atributo de teste	`[data-testid="entrar"]`	✅ Sempre que existir
2	`id`	`#email`	✅ Bom
3	Atributo semântico	`[name="senha"]`, `[type="submit"]`	✅ Aceitável
4	Texto visível	`cy.contains('Entrar')`	⚠️ Quebra se traduzirem
5	Classe CSS	`.btn-primary`	⚠️ Evite
6	Caminho estrutural	`div > div > span:nth-child(3)`	🔴 Nunca

O argumento

Classe CSS é decisão de estilo. Ela muda quando alguém troca o tema, e ninguém avisa o QA. `.btn-primary` funciona hoje e some na próxima atualização do Bootstrap.

Caminho estrutural quebra com uma `div` a mais. Qualquer ajuste de layout derruba o teste.

Atributo de teste existe só para você. Quem o remove sabe que está mexendo em teste. Isso é o contrato.

> DICA
>
> **Se a sua aplicação não tem `data-testid`, peça.** É um pedido barato para quem desenvolve e transformador para a suíte. Enquadre como negociação, não como reclamação: *"se vocês colocarem um atributo nesses cinco elementos, eu paro de abrir chamado toda vez que o CSS muda"*. Essa conversa é uma habilidade de QA sênior, e vale começar cedo.

Quando não há escolha

O ServeRest é um bom exemplo do mundo real: as telas de formulário têm `data-testid`, mas as **tabelas de listagem não têm nenhum** — nem nas linhas, nem nos botões Editar/Excluir, que são idênticos em todas elas.

Quando não há camada melhor, você desce — mas descendo com uma âncora estável:

JAVASCRIPT

```
cy.contains('table.table-striped tbody tr', 'Nome do Produto')  // acha a linha pelo texto
  .contains('button', 'Excluir')                               // acha o botão dentro dela
  .click()
```

Repare no que isso faz: em vez de dizer *"o botão da terceira linha"*, diz *"o botão de excluir **daquela linha que fala do meu produto**"*. Se a ordem das linhas mudar, o teste continua certo.

O mesmo vale para a **mensagem de erro ou de sucesso** do ServeRest: o alerta não tem `data-testid`. Por isso os testes usam `.alert`, que é uma classe CSS (camada 5). Não é contradição com a tabela acima — é descer porque não há camada melhor, e sempre junto de uma asserção sobre o **texto** da mensagem:

```javascript
cy.get('.alert').should('contain', 'Email e/ou senha inválidos')
```

Ferramentas de debugging

```javascript
cy.get('[data-testid="email"]').pause()   // congela o teste para você olhar
cy.get('[data-testid="email"]').debug()   // para no debugger do navegador
cy.log('cheguei aqui')                    // escreve no log do Cypress

cy.get('[data-testid="email"]').then((el) => {
  console.log(el)     // aparece no DevTools
})
```

Como ler uma falha — o método de 4 passos

Este é o conteúdo mais útil do dia. É o que você vai usar toda vez que um teste falhar pelos próximos anos.

1. Leia a primeira linha do erro. Ela diz o que o Cypress esperava e o que encontrou. Não pule para o código.

2. Olhe o print no momento da falha. A tela estava como você imaginava? Muitas vezes a resposta é "não" — havia um modal, um carregando, ou a página nem tinha aberto.

3. Volte um comando no Time Travel. Passe o mouse sobre o comando anterior. O estado antes da falha estava certo? Se já estava errado ali, o problema não é onde você achou que era.

4. Confira o seletor no DevTools. Abra o console e rode:

```javascript
document.querySelectorAll('[data-testid="email"]')
```

Se voltar vazio, o seletor está errado ou o elemento não existe ainda.

> **DICA**
>
> **Esse método é o que separa quem persiste de quem desiste.** Não é sobre ser bom em programação. É sobre ter um roteiro quando a tela fica vermelha.

Erros mais comuns e o que significam

Mensagem	Causa provável
`Timed out retrying... Expected to find element`	Seletor errado, ou o elemento ainda não apareceu
`element is detached from the DOM`	A página se redesenhou entre achar e clicar
`cy.click() failed because this element is being covered`	Modal, overlay ou outro elemento por cima
`cy.visit() failed trying to load`	URL errada ou aplicação fora do ar
`Cannot read properties of undefined`	Erro no JavaScript do teste, geralmente acesso a objeto vazio

EXERCÍCIO · 5 MIN

o instrutor vai mostrar um teste quebrado. Use o método de quatro passos e **apenas identifique** a causa. Não corrija. O objetivo é praticar o diagnóstico.

JAVASCRIPT

```javascript
describe('Teste quebrado — exercício de diagnóstico', () => {
  it('deve fazer login e chegar na home', () => {
    cy.visit('/login')
    cy.get('[data-testid="email"]').type('admin.treinamento@qa.com')
    cy.get('[data-testid="password"]').type('treino123')
    cy.get('[data-testid="entrar"]').click()
    cy.contains('h1', 'Bem Vindo').should('be.visible')
  })
})
```

Resposta — confira depois de tentar. O passo 1 já resolve: a primeira linha do erro diz `Expected to find element: [data-testid="password"], but never found it`. Na tela de **login** o campo de senha é `[data-testid="senha"]`; `password` é o da tela de **cadastro**. O passo 4 confirma: `document.querySelectorAll('[data-testid="password"]')` volta vazio na tela de login.

1.8 Lab do Dia 1

40 minutos.

Crie o arquivo `cypress/e2e/dia1/lab-dia1.cy.js` cobrindo:

1. Login com credenciais válidas de administrador
2. Login com senha inválida, validando a mensagem de erro
3. Cadastro de um novo usuário com e-mail único

4. Tentativa de cadastro com e-mail já existente
5. Logout

Regras:

- Use `beforeEach` para o que se repete
- Prefira seletores da camada 1 ou 2; desça só onde não houver (o `.alert`, como no bloco 1.7)
- Cada teste tem que rodar isoladamente (teste com `it.only`)
- Nenhum `cy.wait` com número

Dica para e-mail único:

```javascript
const email = `usuario_${Date.now()}@qa.com`
```

> **ATENÇÃO**
> **Por que e-mail único?** Se você usar `teste@qa.com`, funciona na primeira vez e falha na segunda: o usuário já existe, e o seu ServeRest guarda os dados de uma execução para a outra. Num ambiente compartilhado é pior — falha também se o colega rodar antes de você. **Teste que depende de dado fixo está quebrado desde o nascimento.**

Credenciais do administrador: `admin.treinamento@qa.com` / `treino123`

> **DICA**
> Lembre: na tela de cadastro o campo de senha é `[data-testid="password"]`. E no item 4, cadastre o seu próprio e-mail antes de testar a duplicidade — nunca use o do admin (veja o aviso do bloco 1.5).

Resumo do Dia 1

O que você leva para casa:

1. **Automatize o que dói se quebrar**, não tudo
2. **Comandos do Cypress não devolvem valores** — o valor chega no `.then()`
3. **Espere condição, nunca tempo.** `cy.wait(3000)` é antipadrão
4. **Cada teste roda sozinho.** Sem herdar estado do anterior
5. **Seletor dedicado a teste** é a primeira escolha, sempre
6. **Leia o erro antes de mexer no código** — o método de 4 passos

Amanhã: você vai testar a API direto, simular respostas do servidor que não conseguiria provocar na mão, e parar de escrever o mesmo bloco de código em doze arquivos.

APÊNDICES

Consulta rápida

A parte da apostila que você vai usar depois do treinamento.

> DICA
> **Estes apêndices crescem com o curso.** Aqui está tudo o que foi visto até hoje. A versão completa vem na apostila do Dia 3.

A Comandos do Cypress por finalidade

Navegação

JAVASCRIPT

```
cy.visit('/login')          // abre uma página (relativa à baseUrl)
cy.reload()                 // recarrega
cy.go('back')               // volta
cy.go('forward')            // avança
cy.url()                    // a URL atual
cy.title()                  // o título da aba
```

Encontrar elementos

JAVASCRIPT

```
cy.get('[data-testid="email"]')    // por seletor CSS
cy.contains('Entrar')              // por texto
cy.contains('button', 'Entrar')    // por texto, dentro de um seletor
cy.get('form').find('input')       // dentro do elemento anterior
cy.get('li').first()               // primeiro
cy.get('li').last()                // último
cy.get('li').eq(2)                 // terceiro (conta do zero)
cy.get('td').parent()              // o pai
cy.get('td').parents('tr')         // o ancestral que casar com o seletor
cy.get('ul').children()            // os filhos diretos
```

Interagir

JAVASCRIPT

```
.click()
.click({ force: true })       // força o clique — use com muita parcimônia
.dblclick()
.type('texto')
.type('texto', { log: false })      // não mostra o valor no log (senhas)
.type('{enter}')              // teclas especiais entre chaves
.clear()
.check()                      // checkbox ou radio
.uncheck()
.select('Opção')              // <select>
.trigger('mouseover')
.scrollIntoView()
```

Trabalhar com valores

JAVASCRIPT

```
.invoke('val')                // chama um método do elemento
.invoke('text')
.its('response.statusCode')   // acessa uma propriedade
.then((valor) => { })         // recebe o valor no callback
cy.wrap(objeto)               // transforma um valor comum em algo "cypressável"
```

Debug

JAVASCRIPT

```
cy.log('mensagem')
cy.pause()
.debug()
cy.screenshot('nome')
```

Linha de comando

TERMINAL

```
npm run cy:open                                  # escrever testes
npm run cy:run                                   # rodar tudo
npx cypress run --spec "caminho/do/teste.cy.js"  # rodar um só
npx cypress run --headed                         # ver o navegador
npx cypress run --browser chrome                 # escolher o navegador
npx cypress run --env chave=valor                # passar variável
```

B Tabela de asserções

Existência e visibilidade

```javascript
.should('exist')
.should('not.exist')
.should('be.visible')
.should('not.be.visible')
```

Conteúdo

```javascript
.should('have.text', 'exatamente este texto')
.should('contain', 'parte do texto')
.should('not.contain', 'este texto não')
.should('have.value', 'valor do campo')
.should('be.empty')
```

Quantidade

```javascript
.should('have.length', 3)
.should('have.length.greaterThan', 0)
.should('have.length.at.least', 1)
```

Estado

```javascript
.should('be.checked')
.should('not.be.checked')
.should('be.disabled')
.should('be.enabled')
.should('have.focus')
```

Atributos e classes

```javascript
.should('have.class', 'ativo')
.should('not.have.class', 'erro')
.should('have.attr', 'href', '/login')
.should('have.css', 'display', 'none')
```

Encadeando

```javascript
cy.get('[data-testid="entrar"]')
  .should('be.visible')
  .and('contain', 'Entrar')      // .and é um .should encadeado
  .and('not.be.disabled')
```

`expect` — dentro de um `.then()`

```javascript
cy.request('GET', url).then((res) => {
  expect(res.status).to.eq(200)
  expect(res.body).to.have.property('quantidade')
  expect(res.body.usuarios).to.be.an('array')
  expect(res.body.message).to.contain('sucesso')
  expect(res.body._id).to.not.be.empty
  expect(res.body).to.deep.include({ nome: 'Fulano' })
})
```

> **DICA**
>
> **`should` ou `expect`?** `should` tenta de novo até passar — use na tela. `expect` roda uma vez só — use dentro de `.then()`, quando o valor já chegou.

C Como ler uma falha: o método de 4 passos

1. Leia a primeira linha do erro. Ela diz o que o Cypress esperava e o que encontrou. Não pule para o código.

2. Olhe o screenshot da falha. A tela estava como você imaginava? Tinha um modal? Um carregando? A página abriu?

3. Volte um comando no Time Travel. Passe o mouse sobre o comando anterior. O estado antes da falha já estava errado? Se sim, o problema não é onde você achou.

4. Confira o seletor no DevTools.

```javascript
document.querySelectorAll('[data-testid="email"]')
```

Vazio? O seletor está errado ou o elemento ainda não existe.

Mensagens de erro e o que significam

D

Mensagem	O que está acontecendo	O que fazer
`Timed out retrying... Expected to find element`	O seletor não encontrou nada no tempo	Confira o seletor no DevTools; veja se o elemento depende de uma requisição
`element is detached from the DOM`	A página se redesenhou entre achar e agir	Refaça o `cy.get()` mais perto da ação, ou espere a requisição terminar antes
`cy.click() failed because this element is being covered`	Tem algo por cima	Feche o modal/overlay. `{ force: true }` é último recurso, e esconde o problema
`cy.visit() failed trying to load`	URL errada ou aplicação fora do ar	Abra a URL no navegador. Se não abrir, rode `npm start` no `serverest-treinamento`
`Cypress.env() was removed in Cypress version 16.0.0`	O projeto está no Cypress 16, e o material é do 15	`npm install cypress@15.21.1 --save-dev`, ou migre para `Cypress.expose()` / `cy.env()` (bloco 2.5)
`Cypress could not verify that this server is running`	O `cypress run` conferiu o `baseUrl` antes de começar e não achou a aplicação	Mesma coisa: suba a loja (`npm start`) e rode de novo
`Cannot read properties of undefined`	Acesso a algo que não existe	Geralmente um `.then()` recebendo resposta vazia, ou fixture com estrutura diferente
`cy.type() can only accept a string or number`	Você passou `undefined` — uma variável que ninguém preencheu	Confira de onde vem o valor: fixture com a chave errada, ou variável que outro teste deveria ter preenchido (bloco 1.6)
Aviso `allowCypressEnv` no terminal	`Cypress.env()` depreciado no Cypress 15	Nada a fazer por ora; substitutos: `Cypress.expose()` e `cy.env()` (bloco 2.5)
`bad option: --smoke-test`	`ELECTRON_RUN_AS_NODE` no terminal	`unset ELECTRON_RUN_AS_NODE` (bash) ou `$env:ELECTRON_RUN_AS_NODE=$null` (PowerShell)

E Seletores do ServeRest

> Conferidos contra a aplicação real. Se algo aqui não funcionar, a aplicação mudou — ela é de estudo e muda sem avisar.

`/login`

Elemento	Seletor
E-mail	`[data-testid="email"]`
Senha	`[data-testid="senha"]`
Entrar	`[data-testid="entrar"]`
Link cadastrar	`[data-testid="cadastrar"]`
Alerta	`.alert`

`/cadastrarusuarios`

Elemento	Seletor
Nome	`[data-testid="nome"]`
E-mail	`[data-testid="email"]`
Senha	`[data-testid="password"]` ⚠️
Checkbox admin	`[data-testid="checkbox"]`
Cadastrar	`[data-testid="cadastrar"]`

`/admin/home`

Elemento	Seletor
Cadastrar usuários	`[data-testid="cadastrarUsuarios"]`
Cadastrar produtos	`[data-testid="cadastrarProdutos"]`
Listar produtos	`[data-testid="listarProdutos"]`
Listar usuários	`[data-testid="listarUsuarios"]`
Logout	`[data-testid="logout"]`

`/admin/cadastrarprodutos`

Elemento	Seletor
Nome	`[data-testid="nome"]`
Preço	`[data-testid="preco"]`
Descrição	`[data-testid="descricao"]`
Quantidade	`[data-testid="quantity"]` ⚠️ em inglês
Imagem	`[data-testid="imagem"]`
Cadastrar	`[data-testid="cadastarProdutos"]` ⚠️ typo da aplicação

Listagens do admin

Não têm `data-testid`. Ancore no texto:

```javascript
cy.contains('table.table-striped tbody tr', 'Nome do Produto')
  .contains('button', 'Excluir')
  .click()
```

Títulos das telas

Rota	`<h1>`
`/admin/home`	`Bem Vindo <nome>`
`/home` (loja)	`Serverest Store`
`/admin/listarprodutos`	`Lista dos Produtos`
`/admin/listarusuarios`	`Lista dos usuários`

As três armadilhas

1. `senha` **é só do login.** No cadastro é `password`.
2. **O botão de cadastrar produto tem typo:** `cadastarProdutos`, sem o segundo "r".
3. `Serverest Store` **não é a home do admin.** É a loja do usuário comum.